

FECAP – FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação

6º Semestre — Turma 6º CCOMP

SMASH

**MODELAGEM FORMAL DA DETECÇÃO DE IMPACTO EM PAREDE
INSTRUMENTADA DE TREINO DE TÊNIS**

Entrega 1 — Teoria da Computação e Linguagens Formais

Projeto 7 — Grupo Smash

Felipe Vallim Soares — 24026060

Guilhermy Mariano Lisboa Garcia — 23025371

Gustavo Oliveira Demetrio — 24026213

Saulo Pereira de Jesus — 24026095

Kátia Milani Lara Bossi

São Paulo

2026

APRESENTAÇÃO DO PROJETO

O **Smash** é um projeto interdisciplinar que transforma uma parede comum em uma superfície instrumentada de treino de tênis. Nove alvos físicos — placas rígidas de MDF apoiadas sobre espaçadores de espuma — são fixados na parede formando uma matriz 3×3 . Colada no verso de cada placa há uma pastilha piezoelétrica que *gera* tensão quando deformada: a batida da bola faz a placa vibrar, a vibração deforma a pastilha e ela devolve um pico de tensão proporcional à força do impacto. Um microcontrolador ESP32 lê os nove canais analógicos, identifica qual alvo foi atingido, mede a intensidade do golpe e transmite o evento por WebSocket a um aplicativo mobile, que o usa para pontuação, progressão e histórico de treino.

Este documento formaliza **parte** do comportamento do sistema: o ciclo de detecção de impacto embarcado no firmware do ESP32, aqui chamado de M1. Esse recorte é a porção do sistema cujo comportamento é genuinamente sequencial, de fase, e portanto a que se presta à modelagem por autômato finito. São usados os conceitos de **autômatos finitos**, **linguagens formais**, **expressões regulares** e **gramáticas** estudados na disciplina, e as seções de 1 a 8 correspondem, na ordem, aos oito itens exigidos no enunciado.

O comportamento modelado não é uma abstração criada para a disciplina: corresponde à máquina de estados já implementada no código do firmware, aqui descrita em termos matemáticos — conjunto de estados, alfabeto de eventos, função de transição, ações de saída, linguagem reconhecida, expressão regular equivalente e gramática geradora.

1 IDENTIFICAÇÃO DOS ESTADOS DO SISTEMA

O Smash é composto por quatro camadas ligadas por um contrato de dados: alvo físico → piezo → firmware (ESP32) → aplicativo mobile → backend e banco. A camada modelada aqui é a do firmware, que governa o ciclo "sinal cruzou o limiar → mede o pico durante uma janela → elege o alvo → entrega o impacto ao laço principal → ignora a ressonância residual". É essa máquina que define o que conta como um impacto.

M1 é um **autômato finito não determinístico com transições ϵ (AFN- ϵ)** de cinco estados, formalmente $Q = \{O, A, M, E, D\}$, sendo **O** o estado inicial e também o único estado de aceitação: o sistema "aceita" uma sequência de eventos sempre que retorna ao estado ocioso, representando um ciclo completo de detecção. As transições ϵ , discutidas na Seção 8.3, são o motivo dessa classificação. Os nomes dos estados são os mesmos usados no tipo enumerado do firmware.

Quadro 1 — Estados do autômato de detecção de impacto (M1).

Estado	Símbolo	O que representa
OCIOSO	O	Estado inicial e de aceitação. O ESP32 varre continuamente os nove canais do multiplexador; nenhum está acima do seu limiar. Nada é medido nem transmitido.
ARMADO	A	Um canal cruzou o limiar. O firmware marca o instante inicial da janela de medição e zera os nove acumuladores de pico. Não aguarda evento externo — a saída ocorre na iteração seguinte do laço principal (transição ϵ).
MEDINDO_PICO	M	Durante a janela de 20 ms, amostra todos os nove canais e atualiza o valor máximo de cada um. É essa comparação que resolve o vazamento de vibração entre placas vizinhas.
EVENTO_EMITIDO	E	A janela se encerrou. Neste estado o firmware executa <code>loudestTarget()</code> , que devolve o canal de maior pico, deriva linha e coluna e preenche a estrutura <code>WallImpact</code> . Nada saiu pela rede ainda. A saída é espontânea (ϵ).
DEBOUNCE	D	Ignora qualquer novo cruzamento de limiar por um intervalo fixo, para que a ressonância residual da mesma batida não seja registrada como um segundo impacto.

2 EVENTOS (ENTRADAS)

Os eventos são as condições observáveis capazes de provocar mudança de estado, e cada um corresponde a uma comparação de limiar ou de tempo presente no código. O alfabeto de entrada de M1 é $\Sigma_1 = \{s, v, q\}$.

Quadro 2 — Alfabeto Σ_1 e condição correspondente no firmware.

Símbolo	Evento	Condição no firmware
s	Cruzamento de limiar	A leitura do ADC de algum canal i ultrapassa <code>LIMIAR[i]</code> — um limiar próprio por alvo, obtido na calibração.
v	Fim da janela de medição	Transcorre a <code>JANELA_PICO</code> (20 ms) desde a abertura da janela. A amostragem dos nove canais termina; a escolha do alvo acontece depois, já no estado <code>EVENTO_EMITIDO</code> .
q	Fim do debounce	Transcorre <code>TEMPO_DEBOUNCE</code> (200 ms) desde a entrada em <code>DEBOUNCE</code> sem que novos cruzamentos sejam considerados.

Vale destacar que *s* é um símbolo único, e não nove símbolos distintos: *qual* canal disparou não altera a fase em que o sistema está, e portanto não pertence à linguagem — pertence à saída, tratada na Seção 4.2.

3 DIAGRAMA DE ESTADOS

A Figura 1 representa graficamente M1: os estados são círculos (OCIOSO com círculo duplo, por ser simultaneamente inicial e de aceitação), as transições são setas rotuladas com o símbolo de entrada, ϵ indica transição espontânea e a seta isolada à esquerda indica o estado inicial. O ciclo é linear: de OCIOSO a DEBOUNCE há um único caminho, e o arco inferior q fecha o ciclo devolvendo o sistema ao repouso.

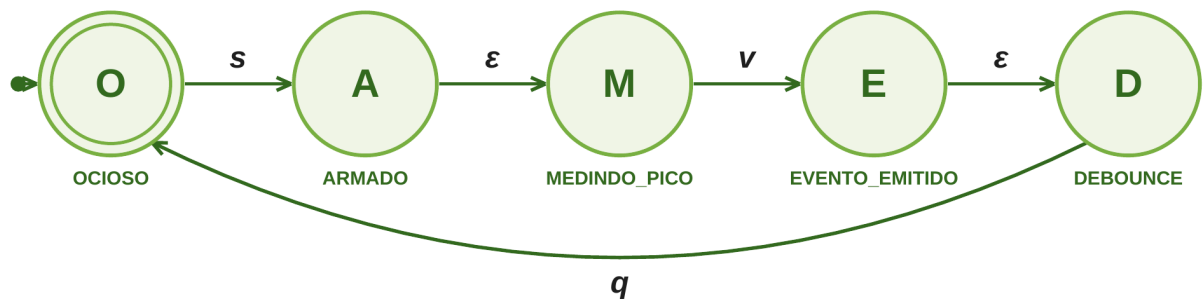


Figura 1 — Autômato de detecção de impacto (M1).

4 TABELA DE TRANSIÇÕES

4.1 Função de transição

A função de transição δ é apresentada de forma explícita a seguir, indicando estado atual, evento de entrada e próximo estado.

Quadro 3 — Função de transição δ de M1.

Estado atual	Evento	Próximo estado	Observação
O	s	A	Um canal passou do limiar: começa uma candidata a batida.
A	ϵ	M	Transição espontânea; a janela de 20 ms já está aberta e a amostragem começa.
M	v	E	A janela de 20 ms apenas se encerra. Nenhum alvo foi eleito ainda.
E	ϵ	D	Transição espontânea: com o <code>WallImpact</code> já preenchido, <code>sensorsPoll</code> retorna <code>true</code> ao laço principal.
D	q	O	Sistema pronto para reconhecer o próximo impacto.

Como duas dessas transições são ϵ , M1 é formalmente um AFN- ϵ . Os pares não listados — por exemplo (O, v) ou (D, s) — são indefinidos e correspondem, na prática, a eventos simplesmente ignorados pelo firmware: é exatamente esse descarte que dá ao debounce a sua função.

4.2 Saídas associadas ao ciclo

A linguagem reconhecida descreve *quando* um impacto acontece, mas não o *que* o sistema produz. A maior parte das ações está nas arestas, à maneira de um **transdutor de Mealy**; a única exceção é o cálculo do alvo vencedor, que está associado ao estado EVENTO_EMITIDO, e não a uma transição. O Quadro 4 explicita a entrada lida e a saída produzida em cada etapa.

Quadro 4 — Entradas lidas e saídas produzidas ao longo do ciclo de M1.

Etapa	Entrada lida	Saída / ação produzida
O → A	Amostra de ADC acima de LIMAR[i]	Marca t_0 , zera os nove acumuladores de pico. Nenhuma saída externa.
A → M	— (espontânea)	Nenhuma saída externa.
M → M	Nove amostras de ADC por varredura	Atualiza $pico[i] = \max(pico[i], leitura[i])$ para $i = 1..9$.
M → E	Fim da JANELA_PICO (20 ms)	Apenas encerra a janela de medição.
em E	— (ação do estado)	<code>loudestTarget()</code> devolve o canal de maior pico; o firmware deriva linha e coluna e preenche <code>WallImpact</code> com alvo, linha, coluna, intensidade e <code>t_ms</code> .
E → D	— (espontânea)	<code>sensorsPoll</code> retorna <code>true</code> , sinalizando ao laço principal que há um impacto pronto.
D → O	Fim do TEMPO_DEBOUNCE (200 ms)	Rearma a varredura; volta a aceitar cruzamentos de limiar.

O envio propriamente dito acontece fora da máquina de estados. Quando `sensorsPoll` retorna `true`, o laço principal chama `linkBroadcastImpact`, e é a camada de protocolo que acrescenta o campo `sessao` e serializa a mensagem JSON transmitida por WebSocket e espelhada no Serial. A separação importa para a modelagem: M1 produz um dado estruturado, não uma mensagem de rede.

```
{
  "alvo": 5,           // canal de maior pico, 1 a 9
  "linha": 2,          // derivado do número do alvo
  "coluna": 2,         // derivado do número do alvo
  "intensidade": 1840, // amplitude do pico, em unidades de ADC
  "t_ms": 42315,       // ms desde o boot do ESP32
  "sessao": "a3f1"     // acrescentado pela camada de protocolo
}
```

Vale registrar que a **intensidade** é uma medida relativa em unidades de ADC, calibrada por alvo, e não uma força absoluta em newtons: serve para comparar batidas entre si dentro do mesmo alvo. A saída do autômato, portanto, é legítima como sinal de comparação, não como grandeza física.

5 LINGUAGEM DO SISTEMA

Considerando $\Sigma_1 = \{s, v, q\}$, a linguagem L_1 reconhecida por M1 é o conjunto das sequências de eventos que levam o sistema do estado inicial O de volta a O, representando zero ou mais ciclos completos de detecção. Cada ciclo é a sequência "svq": limiar cruzado, janela encerrada e debounce concluído.

Exemplos de sequências válidas, aceitas por L_1 :

- ε — sequência vazia: a parede está ligada, mas nenhuma bola foi rebatida ainda.
- svq — um impacto completo: limiar cruzado, janela encerrada, alvo eleito e debounce concluído.
- $svq\ svq\ svq$ — uma sequência de rally com três impactos registrados.

Exemplos de sequências inválidas, não reconhecidas por L_1 :

- $v\dots$ — começa pelo fechamento de uma janela que nunca foi aberta; a partir de O o único símbolo aceitável é s .
- sq — depois de s , o sistema precisa obrigatoriamente encerrar a janela de medição (v) antes de qualquer debounce.
- $svqs$ — termina em M (após o ε de A), que não é estado de aceitação: a janela está aberta, o ciclo está incompleto.
- $svvq$ — duas resoluções de janela para uma única abertura; $\delta(E, v)$ é indefinida.

6 EXPRESSÃO REGULAR

A linguagem L_1 é regular e pode ser expressa pela seguinte expressão regular sobre $\Sigma_1 = \{s, v, q\}$:

$$L_1 = (s\ v\ q)^*$$

A expressão indica que uma cadeia válida é formada por zero ou mais repetições do bloco svq , sempre nessa ordem: todo ciclo começa por s (cruzamento de limiar), avança por v (janela encerrada) e fecha por q (debounce concluído). Não há alternativa nem ponto de escolha — o fecho de Kleene externo é a única fonte de variedade na linguagem, e é ele que expressa o caráter repetitivo de um treino contra a parede.

7 GRAMÁTICA

A seguir apresenta-se uma gramática regular (linear à direita) $G = (V, \Sigma, P, S)$ que gera exatamente a mesma linguagem L_1 :

$V = \{ S, A, B \}$ $\Sigma = \{ s, v, q \}$ S = símbolo inicial

$S \rightarrow s A \mid \varepsilon$

$A \rightarrow v B$

$B \rightarrow q S$

Cada não-terminal corresponde a uma fase do autômato. **S** representa o sistema em OCIOSO, aguardando iniciar um novo ciclo ou encerrar a derivação. **A** representa os estados ARMADO e MEDINDO_PICO combinados, já que a transição entre eles é espontânea e não consome símbolo de entrada. **B** representa, pelo mesmo motivo, EVENTO_EMITIDO e DEBOUNCE combinados.

A derivação de $svq \ svq$, por exemplo, é: $S \Rightarrow sA \Rightarrow svB \Rightarrow svqS \Rightarrow svqsA \Rightarrow svqsvB \Rightarrow svqsvqS \Rightarrow svqsvq$.

Como a gramática é linear à direita, ela gera exatamente a linguagem reconhecida pelo autômato — um resultado esperado, já que gramáticas regulares e autômatos finitos têm o mesmo poder expressivo.

8 JUSTIFICATIVA DA MODELAGEM

8.1 Por que um autômato finito

A escolha de um autômato finito — e não de um autômato com pilha ou de uma máquina de Turing — se justifica pela natureza do problema: o firmware não precisa memorizar informação de profundidade arbitrária nem reconhecer estruturas aninhadas. Basta saber "em qual fase do ciclo" ele está, o que é inteiramente capturado por um número finito e fixo de estados. No código, essa estrutura aparece como um tipo enumerado e uma instrução `switch` que implementa δ a cada iteração do laço principal.

8.2 A fronteira entre estado e saída

Definir o alfabeto foi o ponto mais delicado da modelagem. Tratar os nove alvos como nove símbolos distintos (s_1 a s_9) produziria um autômato com nove ramos paralelos idênticos, sem acrescentar nenhuma informação de comportamento: em todos eles a sequência de fases seria exatamente a mesma. Reconhecer que *qual* alvo foi atingido é **saída**, e não **estado**, mantém a máquina em cinco estados. A mesma decisão vale para a intensidade do impacto: é um dado que o sistema produz, não uma fase em que ele se encontra.

8.3 As transições espontâneas

As transições ϵ foram modeladas dessa forma porque, no código, as mudanças $A \rightarrow M$ e $E \rightarrow D$ ocorrem automaticamente na iteração seguinte do laço, sem depender de nenhuma nova leitura de sensor. Manter esses estados separados, em vez de colapsá-los, tem valor prático: ARMADO é onde a janela é inicializada e EVENTO_EMITIDO é onde o alvo vencedor é calculado — duas responsabilidades distintas, que o grupo quer poder observar separadamente em depuração. É também por causa delas que M1 é classificado como AFN- ϵ , ainda que, do ponto de vista da linguagem reconhecida, os estados ligados por ϵ sejam indistinguíveis — motivo pelo qual a gramática da Seção 7 os combina em apenas três não-terminais.

8.4 Parâmetros e estrutura

Três parâmetros governam o comportamento da máquina: LIMAR[i], próprio de cada alvo, porque a espessura da cola e a fixação variam de placa para placa; JANELA_PICO, de 20 ms, tempo em que o pico de cada canal se forma antes da comparação; e TEMPO_DEBOUNCE, de 200 ms, que cobre a ressonância residual da placa depois da batida.

Nenhum desses valores aparece no autômato, e essa ausência é informativa: recalibrar um limiar ou alongar a janela não altera os estados, as transições nem a linguagem reconhecida. A formalização oferece, assim, um critério prático para separar dois tipos de problema: quando o comportamento observado diverge do esperado por causa de um número, calibra-se; quando diverge porque falta uma fase, a linguagem muda e o firmware precisa mudar com ela.

9 OBSERVAÇÕES REFLEXIVAS

A fronteira entre estado e saída foi a nossa decisão mais importante. Quando reduzimos os nove alvos a um único símbolo s , ficou claro para nós que o autômato não descreve *o que* o sistema mede, mas *em que fase* ele está. Levamos essa distinção para a arquitetura do software: a camada de dispositivo cuida de fase e validação, e a camada de jogo cuida de conteúdo, consumindo impactos já prontos sem saber que existe um piezo do outro lado.

Escrever a tabela de transições nos obrigou a localizar a emissão com precisão. Era tentador dizer que "o evento é enviado quando a janela fecha", mas o código faz isso em quatro tempos: a janela só termina em $M \rightarrow E$; o alvo vencedor é calculado dentro de E ; `sensorsPoll` sinaliza o impacto em $E \rightarrow D$; e só então o laço principal chama `linkBroadcastImpact`, com a camada de protocolo montando o JSON. Foi o formalismo que nos levou a essa precisão — a tabela não aceita "mais ou menos aqui".

O que o modelo não captura também é resultado. Notamos que a linguagem L_1 aceita `svq svq svq ...` indefinidamente, independentemente da qualidade da detecção: uma parede mal calibrada, disparando a cada vibração do ambiente, seria formalmente válida para o nosso autômato. Ou seja, ele nos diz o que é *possível*, e não o que é *bom*. A qualidade da detecção é propriedade estatística da saída — dispersão da intensidade, coerência entre o alvo eleito e o alvo realmente atingido — e fica fora do alcance da linguagem. Descobrir onde a ferramenta para de nos ajudar foi tão útil quanto aprender a usá-la.

10 CONCLUSÃO

A modelagem apresentada demonstra que o ciclo de detecção de impacto do Smash pode ser integralmente descrito por um autômato finito de cinco estados com transições ϵ , por uma linguagem regular, por uma expressão regular equivalente e por uma gramática linear à direita — todas gerando e reconhecendo exatamente a mesma linguagem $L_1 = (svq)^*$.

Mais do que atender ao requisito da disciplina, a formalização rendeu duas coisas concretas ao projeto: uma especificação verificável do firmware, capaz de apontar pares (estado, evento) não tratados antes que virem defeito, e uma descrição precisa de onde cada ação acontece no ciclo. O modelo tende a permanecer estável nas próximas etapas: a persistência via MQTT prevista para a Entrega 2 não acrescenta fases ao autômato, por ser apenas mais um destino da mesma mensagem de impacto que hoje segue para o aplicativo.